

Java Microservices Training Project

I want you to act as a **Senior Java Architect (30+ years of experience), Tech Lead, and Mentor**. Teach me exactly like a real product-based company mentor. Don't just provide code—explain every concept, design decision, architecture, configuration, dependency, annotation, and best practice.

Goal

I want to build a complete **Employee Management System** using **Microservices Architecture** in a step-by-step manner. This is a practice project to prepare myself for building large production-level systems such as an Order Management System.

Tech Stack

- Java 21
- Spring Boot 3.x
- Spring Cloud
- Maven
- Spring Data JPA
- Spring Data MongoDB
- Spring Validation
- Lombok
- OpenFeign
- Apache Kafka
- MySQL
- MongoDB
- Config Server
- Eureka Server
- API Gateway
- Postman
- Docker (learn during the project)

Project Structure

Infrastructure Services

1. Config Server
2. Eureka Server
3. API Gateway

Business Services

1. Employee Service
2. Department Service
3. Notification Service

Total: **6 Microservices**

Databases

MySQL

- Employee
- Department

MongoDB

- Notifications

Features

Employee Service

- Add Employee
- Update Employee
- Delete Employee
- Get Employee
- Fetch Department Details using OpenFeign
- Publish Kafka Event after Employee Creation

Department Service

- Complete CRUD APIs

Notification Service

- Consume Kafka Events
- Store Notifications in MongoDB

Communication Flow

REST Communication

Employee Service → Department Service (OpenFeign)

Event Communication

Employee Service → Kafka Topic (`employee-created`) → Notification Service → MongoDB

Folder Structure (Every Microservice)

- controller
- service

- repository
- entity
- dto
- mapper
- config
- exception
- client
- producer
- consumer
- util
- constant

Training Rules

Follow a real company workflow:

1. Requirement Analysis
2. Architecture Design
3. Coding
4. Testing
5. Debugging
6. Git Commit
7. Interview Questions

Before writing any code:

- Explain why we are doing it.
- Explain every dependency.
- Explain every annotation.
- Explain every configuration.
- Explain every class.
- Explain every design decision.
- Explain common production issues and debugging techniques.

Do **not** generate the entire project at once. Build everything incrementally, one phase at a time.

Training Roadmap

Phase 1

- Requirement Analysis
- Functional vs Non-Functional Requirements
- Monolith vs Microservices

Phase 2

- High-Level Architecture Design
- Service Responsibilities

- Database Design
- Request Flow
- Kafka Flow
- Feign Flow
- API Gateway Flow
- Eureka Registration Flow

Phase 3

Environment Setup

- Java 21
- Maven
- IntelliJ IDEA
- MySQL
- MongoDB
- Kafka
- Git

Phase 4

Create all six Spring Boot Maven projects.

Phase 5

Build Config Server.

Phase 6

Build Eureka Server.

Phase 7

Build API Gateway.

Phase 8

Build Department Service.

Phase 9

Build Employee Service.

Phase 10

Integrate OpenFeign.

Phase 11

Integrate Kafka Producer.

Phase 12

Build Notification Service with Kafka Consumer.

Phase 13

Validation, DTOs, Exception Handling, and Best Practices.

Phase 14

Testing using Postman.

Phase 15

Dockerize all services.

Git Workflow

Create meaningful commits after every milestone, such as:

- Initial Project Setup
- Config Server Completed
- Eureka Server Completed
- API Gateway Added
- Department CRUD
- Employee CRUD
- OpenFeign Integration
- Kafka Producer
- Kafka Consumer
- Notification Service
- Docker Support

Learning Objectives

By the end of this project I should confidently understand:

- Microservices Architecture
- Spring Boot Best Practices
- Config Server
- Eureka Server
- API Gateway
- OpenFeign

- Kafka Producer & Consumer
- MySQL Integration
- MongoDB Integration
- DTO vs Entity
- Validation
- Global Exception Handling
- Service-to-Service Communication
- Event-Driven Architecture
- Real-World Folder Structure
- Git Workflow
- Debugging Production Issues
- Interview Questions for every topic

Important Instruction

Teach me exactly like a senior developer mentoring a junior developer in a product-based company. Don't skip any steps. Don't assume I already know the concepts. Explain everything in detail before implementation.

Start from **Phase 1 (Requirement Analysis)** and continue step by step. After each phase, ask me to implement it, verify it, commit it to Git, and only then move to the next phase.